



PDF Download
3731569.3764855.pdf
29 March 2026
Total Citations: 1
Total Downloads: 4154

Latest updates: <https://dl.acm.org/doi/10.1145/3731569.3764855>

RESEARCH-ARTICLE

METIS: Fast Quality-Aware RAG Systems with Configuration Adaptation

SIDDHANT RAY, The University of Chicago, Chicago, IL, United States

RUI PAN, Princeton University, Princeton, NJ, United States

ZHUOHAN GU, The University of Chicago, Chicago, IL, United States

KUNTAI DU, The University of Chicago, Chicago, IL, United States

SHAOTING FENG, The University of Chicago, Chicago, IL, United States

GANESH ANANTHANARAYANAN, Microsoft Corporation, Redmond, WA, United States

[View all](#)

Open Access Support provided by:

[The University of Chicago](#)

[Microsoft Corporation](#)

[Princeton University](#)

Published: 12 October 2025

[Citation in BibTeX format](#)

SOSP '25: ACM SIGOPS 31st Symposium
on Operating Systems Principles
October 13 - 16, 2025
Seoul, Republic of Korea

Conference Sponsors:
SIGOPS

METIS: Fast Quality-Aware RAG Systems with Configuration Adaptation

Siddhant Ray
University of Chicago

Rui Pan
Princeton University

Zhuohan Gu
University of Chicago

Kuntai Du
University of Chicago / TensorMesh

Shaoting Feng
University of Chicago

Ganesh Ananthanarayanan
Microsoft

Ravi Netravali
Princeton University

Junchen Jiang
University of Chicago / TensorMesh

Abstract

RAG (Retrieval Augmented Generation) allows LLMs (large language models) to generate better responses with external knowledge, but using more external knowledge causes higher response delay. Prior work focuses either on reducing the response delay (e.g., better scheduling of RAG queries) or on maximizing quality (e.g., tuning the RAG workflow), but they fall short in systematically balancing the *tradeoff* between the delay and quality of RAG responses. To balance both quality and response delay, this paper presents METIS, the first RAG system that *jointly* schedules queries and adapts the key RAG configurations of each query, such as the number of retrieved text chunks and synthesis methods. Using four popular RAG-QA datasets, we show that compared to the state-of-the-art RAG optimization schemes, METIS reduces the generation latency by 1.64 – 2.54× without sacrificing generation quality.

CCS Concepts: • Information systems → Information systems applications; • Computer systems organization → Real-time system architecture.

Keywords: RAG systems, LLM inference, Scheduling

ACM Reference Format:

Siddhant Ray, Rui Pan, Zhuohan Gu, Kuntai Du, Shaoting Feng, Ganesh Ananthanarayanan, Ravi Netravali, and Junchen Jiang. 2025. METIS: Fast Quality-Aware RAG Systems with Configuration Adaptation. In *ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP '25)*, October 13–16, 2025, Seoul, Republic of Korea. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/3731569.3764855>



This work is licensed under a Creative Commons Attribution 4.0 International License.

SOSP '25, Seoul, Republic of Korea

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1870-0/2025/10

<https://doi.org/10.1145/3731569.3764855>

1 Introduction

Retrieval-augmented generation (RAG) is a popular LLM inference technique that augments an LLM inference query with relevant text chunks, or “context”, *retrieved* from a large corpus.¹ **RAG systems**, which include retrieval and LLM inference,² have found many use cases in QA tasks, personal assistants, chatbots, and LLM-powered search [10, 67]. While RAG can enhance the quality (accuracy and relevance) of LLM-generated responses [7, 58, 63, 97, 103], RAG queries are inherently slow as they need more compute and memory resources to process the long input context to answer a query [6, 15, 45]. Thus, it is essential to balance *high response quality* and *low response delays* in RAG inference systems.

Past research efforts have optimized RAG, regarding either response quality or response delay, but they fall short in optimizing the **quality-delay tradeoffs** of RAG. RAG queries have an associated *RAG configuration* which describes how and how much data to input for the query (more in §2) [78, 85, 89]. One line of prior work focuses on reducing response delay through better query scheduling (e.g., GPU allocation and inference batching) for RAG queries [2, 48, 49, 76, 82], without adapting the RAG configuration themselves. An alternate line of work focuses on maximizing generation quality by tuning the configurations of RAG queries [35, 83, 89], but this is often done at the cost of longer response delay.

The RAG configuration *simultaneously* affects generation quality and response delay (e.g., retrieving too many chunks for a simple RAG query may unnecessarily inflate delay without increasing quality). Unlike traditional data queries (e.g., SQL) which specify the inputs and operators, RAG queries are inherently *under-specified* as they consist of a text query written in natural language [28, 35, 62, 69] and do not directly specify the exact RAG configuration of its execution.

¹RAG vs. long-context models is an active field of research, with the industry widely deploying RAG for its task-focused model inference quality and better resource-sharing capabilities [73].

²Though RAG sometimes refers to the retrieval step in this work, a RAG system includes both retrieval and LLM inference based on the retrieved texts, and we aim to optimize the whole pipeline.

Moreover, *multiple* configuration knobs can influence the delay-quality tradeoffs. For instance, besides how many chunks to retrieve, *how* to use them in the LLM’s input involves two design choices—should the chunks be processed by the LLM jointly, or should the chunks be summarized first before being fed into the LLM together (and how long should a summary be). Recent works also attempt to tune RAG configuration [35, 83], but they focus on either tuning individual knobs or maximizing quality at the cost of higher delay. However, tuning configurations across multiple knobs quickly hits a *prohibitive* combinatorial space (more in §3) and requires optimizations to reduce the search cost.

What’s more, the RAG configuration should be tuned *jointly* with scheduling. Consider two configurations: *A* feeds all retrieved text chunks in one LLM input, and *B* summarizes first each chunk with an LLM and then feeds the summaries to an LLM input for a final generation. While *A* (which calls the LLM once) is seemingly faster than *B* (which calls the LLM multiple times), *A* could be slower as it requires more GPU memory than *B* and thus could be delayed in the scheduler queue. Without making batching and configuration selection jointly, it would be difficult to avoid such pitfalls.

Finally, the impact of RAG configurations on quality-delay tradeoffs also varies *significantly* with queries. For example, to answer “In which country is the Kimbrough Memorial Stadium located?”, the RAG may retrieve and analyze one text chunk about the stadium. In contrast, to answer “Compare NVIDIA’s operating cost over the first three quarters of 2024 and identify the highest one”, the RAG may need multiple chunks, each containing the quarter’s operating cost, and process these chunks jointly, instead of reading them separately. The above examples illustrate queries differ in *complexity* (more in §4), leading to needing different configurations per-query for optimal quality-delay tradeoffs. Empirically, we show that picking RAG configuration *per-query* achieves 12 – 15% higher quality and 2.5 – 3× lower delay than using any fixed configuration across all queries in a dataset (§5). Thus, RAG configurations should be adapted on a *per-query* basis.

Yet, existing RAG systems, which hand-pick a *static* configuration offline based on a few example queries [1, 21, 42, 91], lose out on quality or response time.

This paper presents METIS, the *first* RAG system that adapts multiple configuration knobs on a per-query basis and jointly makes configuration selections and scheduling decisions (*i.e.*, which LLM inference in a batch) to optimize the delay-quality tradeoffs for RAG.

As this would require solving a joint combinatorial problem for every query, which can be prohibitively expensive (§3), METIS tackles the challenge with a two-step approach.

First, METIS *prunes* the massive configuration space for each received query to a smaller yet promising one that contains configurations that likely yield high-quality output for the given query. Specifically, METIS uses a separate LLM to estimate the query’s *profile*, including how many pieces of

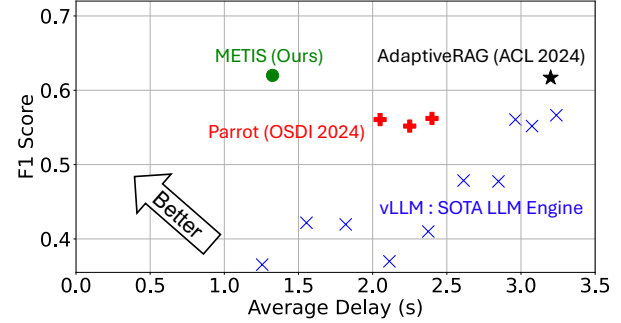


Figure 1. Performance of METIS on the KG RAG FinSec [55] dataset compared to the baselines. Full results shown in §7.

information are required to answer the query and whether joint reasoning is likely required across these pieces of information (more in §4.1). The intuition of the query profiles is that they can effectively filter out undesirable RAG configurations. For the earlier query example “Compare NVIDIA’s operating cost over the first three quarters of 2024 and identify the highest one”, the estimated profile would suggest that it involves at least three separate pieces of information, so the number of chunks (one of the configuration knobs) should be at least three. It should be noted that the LLM-based profiler is an extra overhead in METIS, but fortunately, its input only contains the RAG query itself and the metadata of the RAG database, which are orders of magnitude shorter than the long contexts in RAG, so the estimation can be relatively *fast*, about 1/10 of the delay of the execution of the RAG query.

Using the narrowed configuration space, METIS reduces the RAG response delays by *jointly* deciding the per-query configuration and query scheduling based on available resources (§4.3). The insight is that within the pruned configuration space, the scheduler can make optimal configuration decisions without exploring the original, large configuration space and the implications on quality.

In short, METIS’s two-level design *loosely* decouples the problem into (1) pruning configuration space to a smaller yet promising range of configurations, which focuses solely on keeping the accuracy high, and (2) jointly optimizing configuration (within the narrowed range) and scheduling to optimize response delay by choosing configurations which best-fit into the GPU memory.

We evaluate METIS across four RAG datasets with diverse query profiles (*e.g.*, reasoning vs. domain-specific QA). Figure 1 shows a preview of our results. Our key takeaways are as follows. When achieving the same or higher quality than the baselines, METIS reduces the response delay by 1.6–2.8× compared to the latest vLLM (a state-of-the-art serving engine), Parrot (the latest LLM query-scheduling method), as well as AdaptiveRAG (the latest RAG configuration-tuning method). METIS also achieves 1.8 – 4.5× higher throughput compared to these baselines when achieving the same response delay and same/higher quality.

The general concept of using LLMs to guide system tuning is not exactly new [65, 94], but our key contribution lies in applying the concept to RAG systems, through joint scheduling with resource-aware configuration selection, leading to significantly better resource sharing (§4.2, §4.3). METIS is the first work which (a) shows the importance of tuning multiple RAG knobs; (b) profiles multiple knobs and adapts them simultaneously; and (c) is the first LLM system to introduce resource-quality tradeoff in its RAG decisions.

2 RAG systems and configurations

As an LLM often does not have domain-specific or up-to-date knowledge, LLM applications commonly employ RAG to supplement LLM inference with external knowledge to generate high-quality responses. Despite the growth of model context length, using RAG to pinpoint the relevant context is still significantly cheaper in terms of *resource cost* (GPU requirement), *latency*, and *memory consumption* (KV Cache size). For general-purpose QA pipelines, RAG is cost-efficient with retrieving targeted chunks based on semantic similarity to the query. Using LLMs with long-context documents in contrast has much higher GPU memory usage and delay [47, 49, 77].

Before processing queries, a RAG system organizes background documents by splitting them into chunks (each with a fixed number of tokens), embedding each chunk using models like Bert [12, 19], and storing the embeddings with the chunks in a vector database.

Processing a RAG query involves two main steps:

- **Retrieval:** The RAG system retrieves one or more relevant context chunks from the database by comparing the query’s embedding, (using the same embedding model as for database indexing), with the stored embeddings.
- **Synthesis:** After retrieving the relevant chunks, the RAG system combines these chunks and the RAG query to form a single/multiple LLM call(s) to generate the response.

Retrieval is computationally lightweight and much faster than synthesis ($> 100\times$), so the response delay is typically dominated by the synthesis step [96].

RAG configuration: This work focuses on optimizing three configuration **knobs**, illustrated in Figure 2, which are derived from key design questions that affect RAG performance in terms of response delay and quality:

- **How many chunks to retrieve** (`num_chunks`): The number of context chunks directly affects the delay of the synthesis step, with more computation needed to process the longer sequences with more chunks. In the meantime, retrieving too few chunks risks low response quality if the retrieved chunks do not contain enough useful information.
- **How to synthesize** (`synthesis_method`): If the LLM should read the chunks separately, RAG uses the LLM to generate one answer for the query using each chunk separately and picks the output with the highest confidence, which

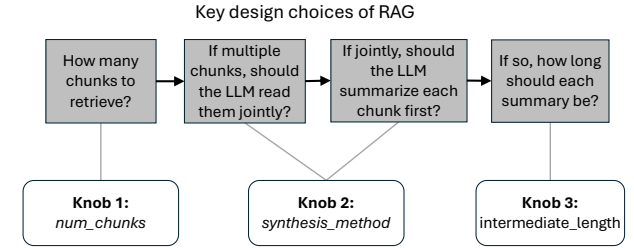


Figure 2. The configuration knobs adapted by METIS are derived from key design choices of RAG systems.

is called `map_rerank`. This often incurs the least computation but can cause low quality if the useful information is scattered in different chunks, in which case the LLM should read the chunks jointly. The RAG system can feed these chunks in the LLM input directly by concatenating them within a single prompt (called `stuff`) or to create a shorter summary for each chunk first before feeding the summaries and the query into the LLM to generate the final response (called `map_reduce`). `stuff` needs less computation than `map_reduce`, but risks degraded output quality for long inputs due to the lost-in-the-middle problem [51].

- **How long is each summary** (`intermediate_length`): Finally, if the LLM produces the summary for each chunk based on the user query, the length of each summary greatly affects the quality and response of `map_reduce`—shorter summaries yield lower delay but also risk not feeding enough information to the final LLM inference.

In this work, while we focus on universal RAG knobs which affect quality and delay common to *all* RAG systems, METIS can be extended to other tunable knobs (e.g., some RAG system may dynamically choose the embedding model, retrieval index or serving LLM). METIS’ design is extensible to any RAG configuration knob based on the query profile.

Performance metrics: We evaluate the performance of a RAG system using two metrics:

- **Response quality** calculates the F1 score of the generated response against the ground truth. The F1 score is the harmonic mean of precision (# correctly generated words) and recall (# of correct words successfully generated). This metric is widely used in prior works [10, 75, 78].
- **Response delay** measures the time elapsed from when the RAG system receives a RAG request to when it completes generating the response.

Next, we show that these knobs need to be properly tuned on a per-query basis to achieve optimal tradeoff between quality and delay in §3.

3 Towards better quality-delay tradeoffs

Prior work on RAG either optimizes for lower delay or higher quality, i.e., the first picks static configurations and focuses on reducing the delay by smart scheduling and resource allocation [48, 76, 82] and the second picks RAG configurations

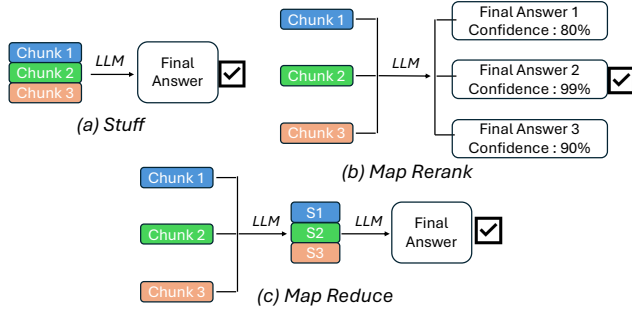


Figure 3. Illustration of different RAG synthesis methods, which have various LLM reasoning capabilities.

to maximize quality without regard to resource usage or delay [35, 83, 89]. For the first time, we explore the potential of optimizing the **quality-delay tradeoffs** for RAG.

To improve the delay-quality tradeoff, our insight is that quality and delay should jointly be optimized in this large tradeoff space created by the choice of RAG configuration knobs. Importantly, the configurations with better quality-delay tradeoffs *vary* significantly across queries.

To showcase this observation, we use three queries from Musique [84], a popular reasoning QA dataset (§7.1).

- **Q1:** “In what county was William W. Blair’s born?”
- **Q2:** “Are Alison Skipper, Diane Gilliam Fisher, and Rachel McAdams from the same country?”
- **Q3:** “When and why did the Voyager 1, the spacecraft that detected storms on Neptune, leave our solar system?”

We chose queries with different natural language complexity and reasoning, Q1 being relatively less complex than Q2 and Q3. Then, we adjust the value of each configuration knob in order to quantify **each knob’s impact on the quality-delay tradeoffs** in each of the queries.

Impact of synthesis method: Figure 4 (a) changes the synthesis method and shows its effect on the quality-delay tradeoff, while keeping the other RAG configuration knobs constant. We vary the synthesis method as map_rerank, stuff, and map_reduce from left to right. The insight is that the optimal synthesis method that strikes the best quality-delay tradeoff (closest to the top left corner) differs significantly across the different queries.

For simple queries like Q1 (green), quality plateaus for more complex synthesis methods (stuff and map_reduce). Because it only needs a single piece of context, map_rerank which processes chunks in isolation suffices, whereas cross-chunk reasoning (stuff and map_reduce) adds undue delay (2×) without improving quality.

For queries such as Q2 (blue) that require *cross-chunk* reasoning, stuff and map_reduce provide significant quality improvements (35% increase) by processing chunks jointly.

For more complex queries, such as Q3 (red), which require even more reasoning and information (why Voyager 1 left

has multiple reasons), methods like map_reduce improve quality (30% increase) by removing unnecessary text in the mapper phase, to help the LLM focus on the relevant content.

Impact of the number of retrieved chunks: Figure 4 (b) fixes the synthesis method (stuff) and shows the impact of the number of retrieved chunks (1-35) on quality and delay.

Simple queries, like Q1 (green), can often be answered using just one or two chunks (needs only birth county). For more complex queries, Q2 (blue) and Q3 (red), increasing the number of chunks (1-15) improves the likelihood of retrieving all relevant context and improves quality.

Blindly retrieving more chunks than necessary risks diluting the relevance of actual important information, due to commonly known problems such as “lost-in-the-middle” [29, 51]. In all three queries, retrieving more chunks beyond a point harms the quality (up to 20% drop) and unnecessarily inflates delay (up to 3×). Hence we have a quality-delay tradeoff where increasing chunks up to a point helps quality but beyond that it increases delay while degrading quality.

Impact of the intermediate output length: Figure 4 (c) shows the impact of our third configuration knob, varying the intermediate output length (1-100) for map_reduce synthesis methods on the quality-delay tradeoff. For simple queries like Q1 (green), short amounts of intermediate length are enough to answer the query (10-20 words). For more complex queries Q2 (blue) and Q3 (red), increasing the amount of intermediate length (70-100 words) provided helps the model with enough information to answer the query.

Overall, we see that RAG queries naturally vary in *complexity*, requiring differing levels of inter-chunk reasoning and varying numbers of context chunks. More complex queries, which require more reasoning and context, benefit from increased LLM computation, which can come at the cost of increased delay. Adding more context chunks helps to a point beyond which it harms the output quality and delay.

Thus, **adapting RAG configuration on a per-query basis is crucial**. Figures 2, 3, 4 *illustrate* tuning most popular RAG configuration knobs, however the tuning extends to more RAG configurations with richer tradeoff spaces (§4.2).

Figure 5 uses queries from two datasets (Musique and QM-SUM, see §7.1) and shows that picking the best configuration for each query (the best configuration is the one with the lowest delay that achieves less than 2% drop than the highest achievable quality) achieves superior quality-delay tradeoff than picking any static configuration for all queries. Choosing the configuration per-query allows up to 3× delay saving compared to static configurations which are the *closest* in quality. Every single static configuration choice that achieves comparable delay has at least a 10% quality drop.

In spite of the potential benefits, per-query configuration adaptation faces challenges that hinder their real-world adoption. Each RAG query comes in plain text with practically no associated RAG configurations. Moreover, the space of

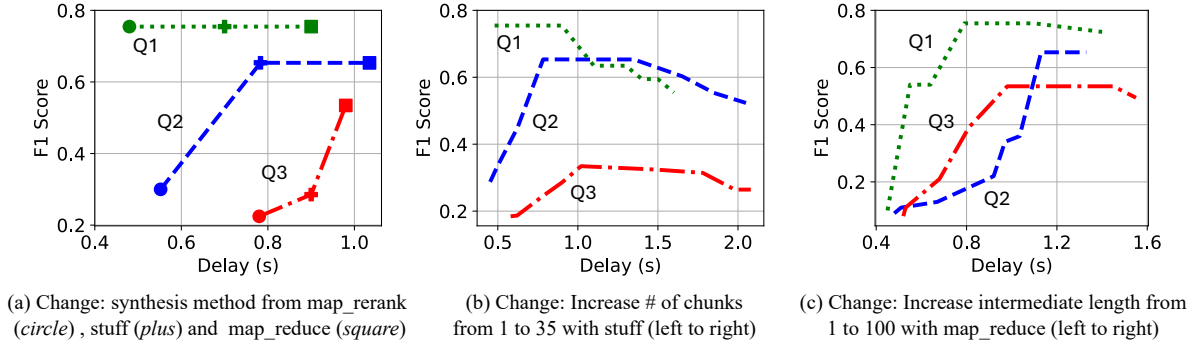


Figure 4. Varying each RAG configuration knob leads to different quality-latency tradeoffs, and these tradeoffs differ across queries (Q1 in green, Q2 in blue, and Q3 in red).

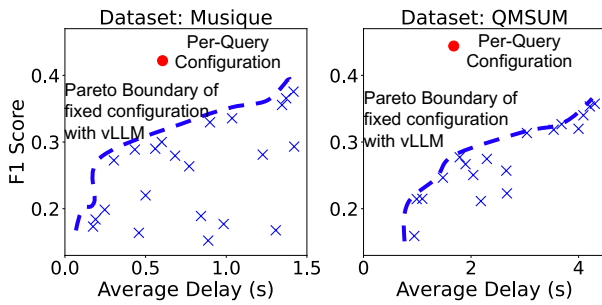


Figure 5. Per-query configuration can achieve significantly better quality-delay tradeoffs across queries compared to every fixed configuration choice.

configurations grows *exponentially* with multiple knobs. For example, for a map_reduce configuration, with 30 values for num_chunks and 50 values for intermediate_length leads to 1500 configurations for a query. Exhaustively profiling all configurations per-query and choosing the best is infeasible.

Alternatively, if we profile periodically, we lose out on the potential configuration selection for *each* query, as variance in query profile leads to different quality-delay tradeoffs. Profiling cost is also *prohibitively* expensive as the LLM needs to be run with many synthesis methods, number of chunks *etc.*, which require high GPU usage. Additionally, the delay of profiling can be $\sim 100\times$ the inference delay due to multiple LLM calls during profiling. Online RAG queries have stringent requirements for GPU resource usage and end-to-end delay [76, 82]. This makes it hard to systematically decide what an optimal per-input configuration should be.

To truly achieve the benefit of per-query configuration adaptation, we need a *smart* system to *drastically* reduce to a useful configuration space, in a *fast* and *cheap* manner.

Finally, in the emerging space of agentic and reasoning RAG, profiling and configuration adaptation remains an integral part due the latency-sensitive nature of these systems. Optimally choosing the configurations, the focus of METIS, remains crucial. We discuss this further in Section 9.

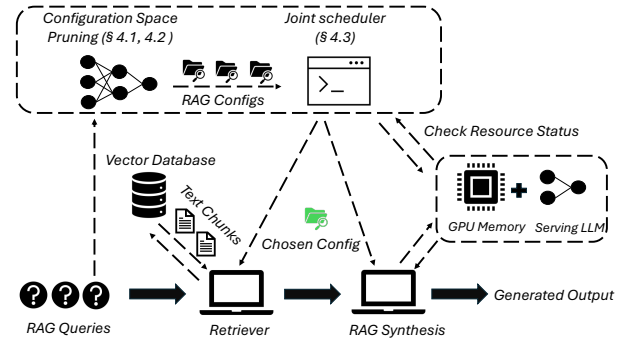


Figure 6. METIS consists of a RAG controller which performs configuration space pruning and joint scheduling.

4 METIS: Enabling per-query configuration adaptation for RAG

We present METIS, a novel system for serving RAG queries focusing on high generation quality and minimal delay. METIS is a RAG controller (Figure 6) with two main components:

- *Pruning configuration space:* We estimate each query’s profile (§4.1) and reduce the RAG configuration space to a smaller yet promising one that still yields high generation quality (§4.2) (leading to a 50-100 $\times$ reduction).
- *RAG scheduler:* Within the pruned configuration space for the query, METIS’ scheduler chooses the best configuration for the query to achieve the best quality-latency trade-off based on the available system resources (§4.3).

Once the configuration is chosen, the METIS’ executes the query using the chosen configuration—retrieving the selected number of chunks and uses the selected synthesis method to feed into the LLM’s input.

4.1 Estimating a query’s profile

Query profile: To choose the correct RAG configurations, the first step of METIS is to create the profile of the query (as we see in Figure 7) by querying an LLM (we call this LLM *query profiler*). We ask the query profiler to estimate four high-level dimensions for each query.

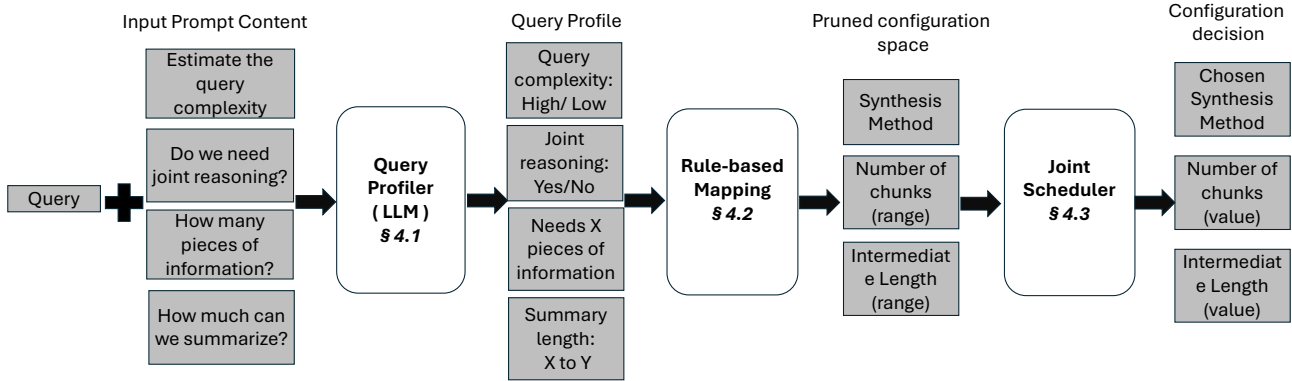


Figure 7. METIS RAG configuration selection workflow.

- *Query complexity* refers to the intricacy of the query itself. Queries with less complexity are more like simple yes/no questions, while queries with high complexity are more like why questions, which require deeper reasoning than yes/no questions. As a result, it requires more LLM computation to correctly answer complex queries. The output for this dimension is binary “High/Low”
- *Joint reasoning requirement* describes whether multiple pieces of information are needed to answer the query. Even relatively simple queries may require joint reasoning (e.g., checking whether the annual income from two years is the same). The output for this dimension is binary “Yes/No”
- *Pieces of information required* refers to the distinct, standalone pieces of information required to fully answer the query (e.g., the annual income from how many years is required to draw the trend of annual income). The output for this dimension is a number from 1-10.
- *The length of the summarization*: If the query is complex and needs a lot of different information, it is often necessary to first summarize the relevant information chunks first (to reduce the noise inside these chunks) and then generate the final answer from these summaries. The output for this dimension is a number from 30-200.

METIS is not the first to use query profile as a metric for deciding RAG configurations, it extends upon methods like AdaptiveRAG [35] which have used LLM’s to estimate query profile but they only focus on one dimension (the number of chunks to retrieve). However METIS is the first LLM system to introduce resource-quality tradeoff in its RAG decisions with *multiple* RAG configurations. We show this tradeoff in our experiments, along with the impact of each dimension on the overall improvement, in Section 7.

Why the query profile *could* be estimated: Estimating the aforementioned query profile is feasible, not only because of the reasoning power of LLMs³ in analyzing natural

language queries, but also because we provide sufficient information to the LLM-based profiler. METIS feeds the profile estimator with not only the query, but also a *metadata* of the database that contains the background document.

The metadata is a short description about the type of content in the database and its data size (chunk_size). Specifically, we use a single-line summaries already attached to the original source datasets as the metadata of the dataset. For example, the metadata for the KG RAG Finsec’s database [55] contains quarterly financial reports and questions of Fortune 500 companies with a chunk_size of 1000. It describes the content topics of the chunks with information such as revenue growth indicators, product release information, sales *etc.*,. When presented with a query on financials of such a company, the LLM can use the metadata to decide questions like how much to summarize/how much reasoning is required. We give details on the prompt and the intuition to generate metadata for new datasets in Appendix §A.

The profiler uses an expressive model, as it only sees the input query and the dataset metadata without the whole context required for the RAG query. Based on the profiling and the available resources, a much smaller model works for inference as RAG uses the retrieved context, rather than model weights, to answer the question. The profiler, though a larger LLM, is cost-bounded, as the input is much smaller (~100-1000×) [45] compared to the retrieved context.

It is important to acknowledge that for highly under-specified queries, it is hard for any model (even human) to reasonably estimate the query’s profile. For an example query “Compare current US Stock Market trends,” the query profile here does not provide enough information (e.g., how many years should the trend be derived from). To answer such highly under-specified queries, more information about the dataset will unlikely help.⁴

Moreover, we observed that extra information does not significantly improve the profiler’s estimates. For instance,

³We have tested both GPT and Llama models as the profile query-profiler, and they yield similarly impressive results (§7).

⁴Maybe some chat history from the same user will help, but that is beyond the scope of this work.

Algorithm 1: Rule based mapping algorithm

Input: *Query complexity, Joint reasoning required*
Input: *Pieces of information, Summarization length range*
Result: *synthesis_method, num_chunks, intermediate_length*

```

1 if Joint reasoning required == "no" then
2   | synthesis_method = map_rerank
3 else
4   | if Query complexity == "low" then
5   |   | synthesis_method = stuff
6   | else
7   |   | synthesis_method = stuff, map_reduce
8 num_chunks = [Pieces of information, 3× Pieces of information]
9 intermediate_length_range = Summarization length range

```

in theory, it helps to know the embedding algorithm used by RAG. Yet, the embedding models perform similarly overall across queries and datasets under our consideration. This explains their limited contribution to the profiler, though more future work is needed to understand the wider implications.

4.2 Mapping query profiles to RAG configurations

After METIS obtains the query profile using the LLM, it performs rule-based mapping to generate values for RAG configuration knobs (e.g., *synthesis_method* etc. introduced in §2). based on the query profiler’s outputs.

How we map and why the profile helps: To understand the role of query profiles, consider the following examples:

- “Who is the current CEO of NVIDIA?” This query is not complex and does not require joint reasoning. Due to the query being simple with no reasoning required and one piece of information (name of CEO).
- “Which month had the highest NVIDIA’s stock price the six months from January to June 2024?” This query is simple but still needs to read information jointly, specifically six pieces of information (stock price for every month)
- “What are the reasons for NVIDIA’s month-on-month stock price change from January to June 2024” This query is complex and needs to read multiple pieces of information jointly (stock prices, reasons for change etc.) As multiple reasons need to be analyzed here, summarizing all of the information first helps narrow down to relevant information and perform clearer reasoning (why the prices changed).

Algorithm 1 outlines the rule-based mapping process. This mapping is significantly helpful, it improves upon raw profiler outputs and converts them to usable RAG configurations. It reduces the cost of the profiler LLM by restricting it to provide short binary decisions only.

We decide the range of *synthesis_method* selections based on two of the profile dimensions estimated in §4.1, i.e., the “Query complexity” and the “Joint reasoning requirement”. Simple queries that don’t need any reasoning can answered with *map_rerank* while queries that require joint reasoning need *stuff* or *map_reduce*. We then decide the range of values for *num_chunks* based on the profile dimension of the “Pieces of information required”, i.e., n —specifically, we set the range of *num_chunks* to be 1–3 times of n . We do not directly set *num_chunks* at n , because it (1) gives some leeway for the retrieval logic (e.g., typically Bert-embedding-based)⁵ to find necessary information, and (2) provides the room for the scheduler to select the configuration that fits in available memory. Finally, we get the *intermediate_length* range from the “summary length” estimate, which is already a value range (derived from the query, metadata and chunk size).

Algorithm 1 is *central* to METIS’ design to reduce to the space to our useful RAG configurations and this is *extendable* to other RAG configurations. For instance, a particular RAG pipeline might use an external re-ranker [23, 57], query re-writer [39, 56] or perform an external web-search [79] along with database retrieval. The mapping algorithm can map the profiling LLM’s output (e.g., of *Query complexity*) and be used to guide such decisions for these newer RAG configurations.

Finally, it is important to note that the concept of METIS belongs to an active research trend in the ML and systems community that leverages LLM outputs and mapping functions to guide real system decisions and optimizations, an example of which is *LLM routing* [13, 32, 61, 64]. While current LLM routers use trained LLMs to map decisions from query complexity to only choose from families of inference models (outside the realm of RAG), we differ by mapping the output to the configuration knob we run for RAG queries.

Like these prior efforts, METIS is a heuristic to best utilize the LLM-generated information to guide system optimizations. While it demonstrates remarkable improvement in practice, more work will be needed to complement it for better interpretability and robustness.

4.3 Joint configuration-scheduling adaptation

Once provided with the narrowed range of each RAG configuration knob (*synthesis_method*, *num_chunks* and *intermediate_length*), we need to choose a RAG configuration, which is aware of the current system resource (GPU memory). If we pick configurations which do not fit in current memory, it will lead to additional queuing delay waiting for the GPU memory to free up.

We have METIS’s pruned configuration space where the quality is high, we now focus on choosing the best configuration which fits in memory, without focusing on quality.

⁵A typical RAG retriever will retrieve 2-3× more chunks than minimally required to provide sufficient information for the LLM inference [24, 60].

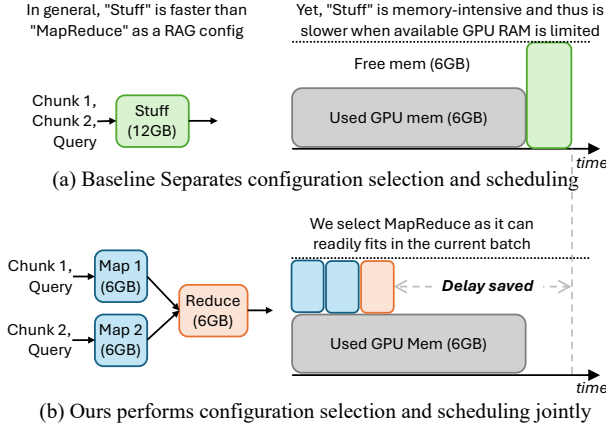


Figure 8. METIS joint schedules RAG configurations with available GPU memory (chosen example - map_reduce)

Why we need to choose the scheduling jointly: We motivate the need for joint scheduling along with the RAG configuration choice in Figure 8.

Consider a setup where we tune only one RAG configuration knob of `synthesis_method`. Other knobs `num_chunks` and `intermediate_length` are fixed at 20 and 100 respectively. Let's assume both `stuff` and `map_reduce` are present in the pruned space. For the scheduling knob, we consider the amount of GPU memory available for the current batch.

Consider a baseline system which separates the joint decision from the scheduling and picks only the RAG configuration knob (`synthesis_method`). It chooses the `stuff` configuration knob as it has lower compute requirement, so given enough memory it should be fast.

The baseline system in Figure 8 (a) does not consider other jobs in the system and does not evaluate the amount of available resource to make its scheduling decision. Due to its long input length with 20 chunks, `stuff` turns out to be memory-intensive. If the available GPU memory is low, `stuff` doesn't fit in memory and needs to be queued. This ends up with `stuff` being slow.

Jointly considering the available GPU memory with choosing the RAG configuration knob avoids this pitfall. For example, in Figure 8 (b), if the original configuration was `stuff`, METIS can choose to use `map_reduce` (based on the current GPU memory available).

By doing so, METIS can start putting the mappers which fit in memory, into the current `running_batch` of requests which fits in the GPU. While `map_reduce` requires more compute, in this case, it benefits from being able to start execution much faster, as some of the mappers fit in memory.

METIS does not need to wait for the GPU memory to free up and changes the configuration aware of system resource, to save delay and achieve a better quality-delay tradeoff.

Jointly choosing the configuration knobs: METIS first provides us with a pruned range of configurations. A *straw-man* solution is to pick a constant value from the across

queries. (e.g., the median value of the `num_chunks`). While this is better than using one static configuration for all queries, it is still sub-optimal as it does not look at the current system resource availability. This prevents us from exploiting the best quality-delay tradeoff across RAG queries.

We use a *best-fit* algorithm on underlying vLLM's continuous batching to allow for variation in configurations across queries. METIS first computes the GPU memory requirement for the RAG query from the RAG configuration knobs for every configuration in the pruned space. For RAG queries, the memory required (e.g., the KVCache size) is measured from the input token length, parameters of the serving model and the quantization (bytes per token). We keep a small 2% buffer size on top of the measurement to deal with potential OOM crashes. We measure the current *available memory* on the GPU to see what can fit into the current batch.

We then pick the *best configuration* from the pruned space that fits into the GPU. METIS defines the best configuration as the one with overall highest memory requirement, from all which fit in memory. The insight here is that within the reduced range of good quality configurations, higher memory configurations correspond to expensive configurations (e.g. more number of chunks, higher intermediate length). In general, these configurations should lead to *slightly higher quality* in the reduced space. For example, if the pruned space says `num_chunks` is 5-10 and the `synthesis_method` is `stuff` and both 5 or 6 chunks can fit in memory, we choose 6 chunks. We don't pick a configuration that doesn't fit in GPU, so we would never choose more than 6 chunks. If we do that, the system will *queue* the request inflating the delay.

After choosing the configuration that fits into the current `running_batch`, the vLLM engine is optimized to perform *chunked_prefill*. However, even with *chunked_prefill*, it can only offload parts of long prefill of `stuff` requests which do not fit in the current batch and still inflates the queuing delay. Jointly scheduling RAG configurations enables efficient resource usage, which cannot be obtained by only relying on the output of the LLM profiler.

What if none of the configurations fit in the GPU? A main insight for METIS's design comes from the observation that in general, the RAG-specific focused configurations can be *loosely-decoupled* from the scheduling-specific configurations. METIS tries to fit the best possible configurations into GPU memory after it gets the profiler's reduced configuration space. It can sometimes happen that the current GPU memory availability is too low and none of the profiler's configurations fit in the currently available GPU.

METIS handles this by falling back to a cheaper fixed configuration and ignoring the output space of the pruned configurations. As METIS already have access to the query complexity profile, we can pick from cheaper configurations, which would meet the requirement for the current query.

If the query does not require joint reasoning, we pick a `map_rerank` with as many chunks that fit into available GPU

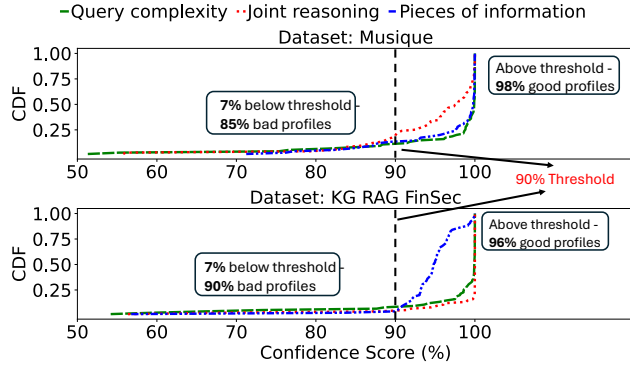


Figure 9. Confidence score threshold for different profiler outputs is used to decide when not to use the profiler output.

memory. If joint reasoning is required, we pick a stuff with as many chunks that fit into memory. METIS does not queue a configuration from outside the pruned range if none fit, but falls back to a fitting configuration just outside the range.

This allows *loose-decoupling* of the RAG configurations into a smaller space and then choosing configurations based on system resource availability. This also allows SLO-based constraints on RAG queries if certain queries have strict budgets on their generation latency.

5 Refinements to METIS

In spite of it all, it is possible for the profiler to (sometimes) fail and in such cases, it is important to detect if METIS’s profiler fails on a query in a fast manner to prevent it from leading to bad RAG configurations. Also it is useful to decide how to provide feedback to METIS to improve.

When is the quality profile reliable? METIS uses LLM to generate the quality profile. Inspired by recent work in use of model confidence [20, 25, 90] as a quality metric, we use confidence scores for METIS’s LLM profiler as to measure the reliability of the profile provided. We obtain the confidence scores from the LLM’s *log-probs* values on the output (the logarithm of the confidence score, which is directly provided with the output with no extra overhead).

We then threshold the confidence score using a confidence score threshold (90% across different datasets) to predict whether the quality profile derived from the quality profiler LLM is actually good (defined as whether the profile can lead to 10% increase in F1-score or 1.5 – 2× reduction in delay or both) or not. Such 90% threshold can be tuned for better performance, and we leave it to future work. From Figure 9, we draw two conclusions. First, *over 93%* of the quality profiles derived from LLM are of high confidence (*i.e.*, over 90%). Further, for those high-confidence profile, over 96% of them are good profiles, meaning that they can be used to improve quality, or reduce latency, or both.

To handle those cases where the quality profile is of confidence score *lower than 90%*, METIS will fall back to the pruned configuration space of recent 10 queries.

How to improve the profiler over time? METIS improves the query profiler LLM by profiling extra feedback prompt to this LLM. We generate this feedback prompt by generating the most accurate output, which is obtained by performing inference on the most resource-demanding configuration (the *map_reduce* configuration with a large number of input chunks (30) and a high value of intermediate length (300 tokens)) and then ask the quality profiler LLM what configuration it should choose based on the query *and* the most accurate answer to that query.

The key insight is that, the most accurate answer to the query provides the quality profiler LLM *extra knowledge* and thus can be used to further improve its decision.

To control the cost of generating feedback prompts, METIS only generates the feedback prompt once every 30 queries and we only keep the *last four* feedback prompts.

The cost of METIS’ LLM quality profiler: For the profiler LLM, we use a larger LLM as compared to the serving LLM (7B parameters). Using this has minimal cost, as METIS *only runs it on the query* itself and in METIS as the query is at least 100× shorter than the context. Using this approach, METIS still saves cost as opposed to using a large LLM for inference (as shown in Section 7). We also show that METIS can use different closed and open-source LLMs as the profiler LLM for pruning and can still provide impressive delay reduction without hurting the accuracy in Section 7.

6 Implementation

We implement METIS in about 2K lines of code in Python on top of the state-of-the-art popular LLM serving engine vLLM [44]. For the profiler used for configuration space pruning, we define a class `LLMProfiler` inheriting OpenAI’s Chat Completion API [66] interface (to invoke GPT-4o) and HuggingfaceAPI [87] (to invoke Llama-3.1-70B) as models to profile the queries.

We use Cohere-embed-v3.0 [4] as a state-of-the-art embedding method. We construct a FAISS [16] index using the `IndexFlatL2` interface and perform L2-distance similarity search with `index.search(query_embedding, top_k)` on the chunk embeddings to retrieve for RAG inference. We use the `LLMChain` interface from Langchain [8] in order to build efficient implementations of multiple synthesis methods.

Finally, we use PyTorch’s [5] library modules support to perform query-level memory profiling and measurement to implement the best-fit scheduling logic and request batching. Particularly, we use `pynvml` to construct `get_free_memory()` with its interfaces of `nvmlDeviceGetHandleByIndex` and `nvmlDeviceGetMemoryInfo` to measure the amount of GPU memory available. We measure the current `num-seqs` and `num-batched-tokens` within vLLM to calculate which configuration can be fit into the current batch, based on the GPU availability and the request’s memory requirement.

7 Evaluation

The key takeaways from the evaluation are

- *Lower delay* : Across 4 task representative datasets for RAG QA, METIS achieves 1.64 – 2.54× lower response delay compared to fixed configurations of comparable quality.
- *Higher throughput* : METIS achieves 1.8 – 4.5× higher throughput than RAG serving systems which use fixed configurations reaching similar quality.
- *Negligible overhead* : METIS’ profiler’s delay is negligible compared to the overall delay of the LLM’s RAG inference.

7.1 Setup

Models and hardware: In RAG, models use external context to answer queries instead of the trained weights (model’s embedded knowledge). The model extracts data from the external context and has stringent serving latency requirements. Hence RAG applications use smaller, instruction-tuned models. We evaluate METIS on a popular models for RAG LLM inference, specifically the fine-tuned version of Mistral-7B-v3 and Llama3.1-70B for additional experiments as they are commonly used in all RAG QA workload serving. All models are fine-tuned such that they can take long contexts (up to 32K and 128K respectively). We apply AWQ-model quantization to both models.

We use an NVIDIA A40 GPU server with 2 GPUs to benchmark our results. The server is equipped with 384GB of memory and two Intel(R) Xeon(R) Gold 6130 CPUs with Hyper-threading and Turbo Boost enabled by default. We use 1 GPU to serve Mistral-7B-v3 and 2 GPUs to serve Llama3.1-70B.

Datasets: We use multiple RAG QA datasets with various query profiles, in order to have task-representative workloads. Table 1 summarizes their input-output statistics.

- Squad [71]: Squad is a single-hop reading comprehension dataset, consisting of questions on articles, where the answer to every question is a segment from the corresponding reading passage.
- Musique [84]: Musique is a multihop QA dataset with reasoning-based questions. It is designated to test LLM’s reasoning ability where one reasoning step critically relies on information from another.
- KG RAG FinSec [55]: KG RAG Finsec is part of a Knowledge Graph family of RAG datasets and focuses on financial domain questions from Fortune 500 companies. This dataset contains quarterly financial reports and queries need to read information for multiple chunks for answering.
- QMSUM [99]: QMSUM is a human-annotated query-based multi-domain meeting summarization benchmark designed to test LLM’s reasoning-based summarization capabilities. This dataset contains multiple meeting transcripts and queries to summarize relevant spans of meetings.

We build a retrieval database database by splitting the queries’ contexts into fixed-sized chunks using Langchain [8]

Dataset	Task Type	Input	Output
Squad	Single hop QA	0.4K - 2K	5-10
Musique	Multihop QA	1K - 5K	5-20
KG RAG FinSec	Doc Level QA	4K - 10K	20-40
QMSUM	Summarization QA	4K - 12K	20-60

Table 1. Input and output length (# of tokens) distributions of the RAG datasets used in our evaluation.

for the database, with Cohere embed-v3.0 [4] embeddings and FAISS [16] L2-distance similarity search in order to retrieve relevant chunks for RAG inference. To simulate a real RAG workload, we choose 200 queries from each dataset, and send them concurrently to METIS using a Poisson distribution with an average arrival rate of 2 per dataset. We report the results per dataset.

Quality Metric: We adopt the following standard metric to measure the generation quality.

- F1-score is used to evaluate the METIS’s serving model’s generated response (defined in §2). It is the most widely adopted metric for evaluating RAG QA tasks [10, 75, 78]

System Metrics: We adopt the following system metrics:

- *Delay* is used to measure the generation response delay of the model for every RAG query. We choose this system metric similar to other RAG serving papers [48, 76, 82]
- *Dollar Cost* is used to measure the lower cost of using METIS’s profiler as compared to using larger serving models with fixed configurations having the closest accuracy.

Baselines: We compare METIS with the following baselines.

- *vLLM*: We serve RAG with vLLM with multiple static configurations across different queries.
- *Parrot**: We implement Parrot’s [48] configuration-based batching. Parrot* does not adapt the configuration per query. We compare with Parrot* using fixed RAG configurations which achieve the closest quality to us.
- *AdaptiveRAG**: We implement AdaptiveRAG’s [35], query complexity-based RAG-configuration selection and choose the configuration which maximizes the F1-score, without considering the system resource cost.

7.2 Overall improvement

Lower delay without sacrificing generation quality: Figure 10 shows METIS achieves delay reduction 1.64 – 2.54× over *AdaptiveRAG** with no reduction in F1-score. Over using fixed configurations of similar delay, served with both *Parrot** and *vLLM*, METIS achieves 12 – 18% higher F1-score.

Higher throughput at lower delay: Figure 11 shows METIS achieves higher throughput compared to fixed configuration baselines when they choose the fixed-config which achieves the closest quality. METIS has dynamic RAG configurations while vLLM and Parrot only allow a fixed configuration across queries. Baselines choose the configuration which achieves the highest average F1-score among all the

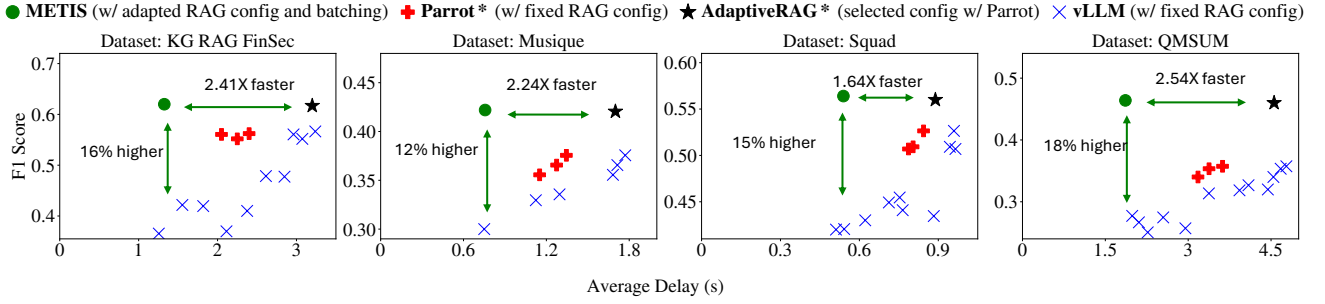


Figure 10. METIS achieves 1.64 – 2.54× lower delay compared to both best fixed configuration baselines and quality-optimized RAG configuration without sacrificing generation quality.

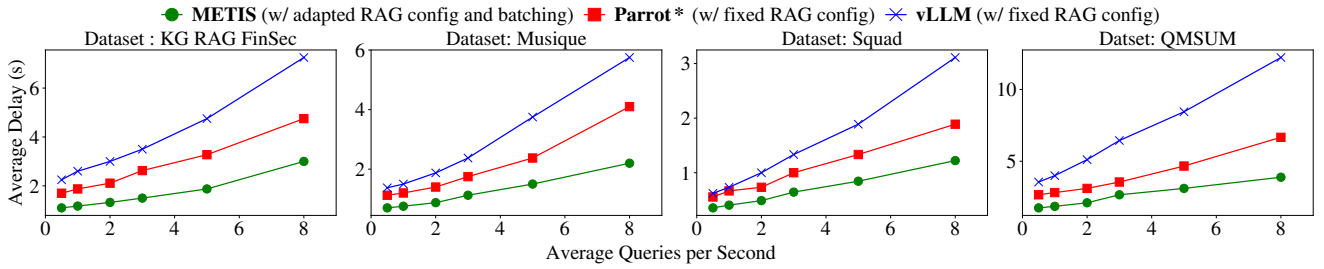


Figure 11. METIS achieves 1.8 – 4.5× higher throughput (at 1.8 seconds) than baselines which use fixed configurations of closest (not higher) quality.

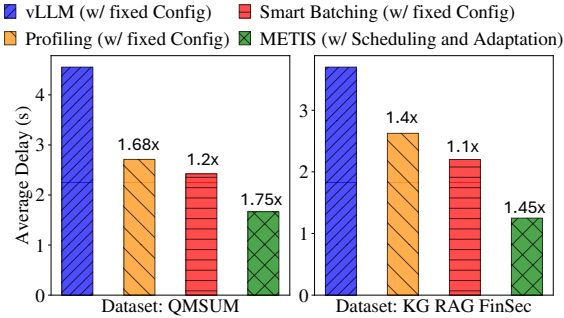


Figure 12. Understanding the delay improvement in METIS

fixed configurations but due to the static nature, every configuration achieves a lower F1-score compared to METIS. Compared to *Parrot** and *vLLM*, METIS achieves 1.8 – 4.5× times higher throughput.

Understanding METIS’ improvement: METIS’s gains come from jointly selecting the configuration based on the available resource, along with performing scheduling. METIS achieves higher quality than the fixed-config baselines as it adapts the RAG-configuration per query. It reduces delay by resource-aware scheduling, making it better than fixed configurations which achieve closest quality.

METIS achieves higher throughput as it is able to adapt configurations based on resource availability as compared to the baselines. Both *Parrot** and *vLLM* schedule fixed RAG-configurations and cannot benefit from delay achieved by

adapting the configuration like METIS. *Parrot** can improve the delay over using fixed configurations with *vLLM* by 1.4 – 1.8× but cannot improve the quality. Compared to *AdaptiveRAG**, METIS achieves lower latency, as *AdaptiveRAG** inflates serving latency without considering the cost of profiling or the cost of the configuration. *AdaptiveRAG** also does not provide an interface to extend to multiple knobs.

7.3 Analyzing the gains from METIS

Delay saving: Figure 12 shows the contribution of every component of METIS. We compare with *vLLM*’s fixed configuration, which achieves the highest quality (blue bar). Using the profiler’s outputs and choosing the median value every time (orange bar), we achieve 1.4 – 1.68× reduction in delay. Next, we see the effect of batching (like *Parrot**), by choosing the median value configuration and batching, we achieve 1.1 – 1.2× reduction in delay. Finally, METIS achieves even greater delay reduction by 1.45 – 1.75× by adapting the configuration based on available GPU memory with batching.

Cost saving: Figure 13 shows METIS (including its profiler) has significant lower dollar cost and higher F1-score, compared to choosing the best fixed configuration, with increasing model complexity. The cost of using a (LLama3-70B) inference model with *vLLM* and a fixed configuration is higher by 2.38× times while also having a lower F1-score of 6.5% times across datasets. Even more powerful inference models like GPT-4o fail to achieve the same F1-score with fixed configurations but have a much higher cost of 6.8×.

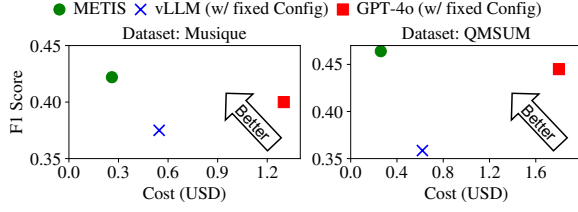


Figure 13. Even with increasing the inference model size, fixed configurations have 2.38 – 6.8 $\times$ higher cost and lower quality compared to METIS.

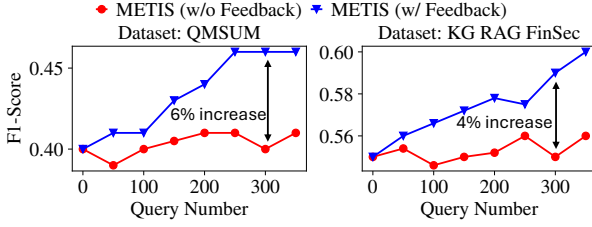


Figure 14. Improvement for METIS using feedback from the output helps improve the F1-score by 4 – 6%.

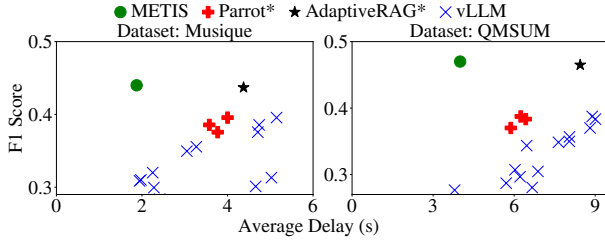


Figure 15. METIS achieves lower delay by 2.1 – 2.4 $\times$ at the same quality even with a larger inference LLM.

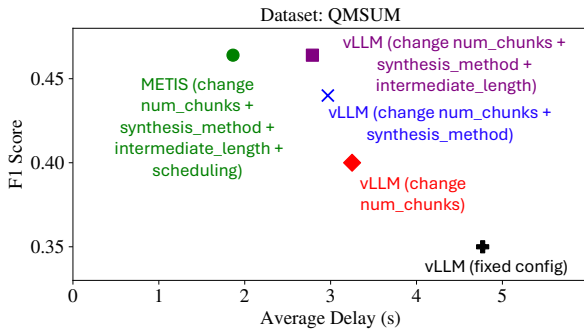


Figure 16. Breakdown analysis: By tuning more knobs in METIS, we can see better quality-delay tradeoffs.

Profiler feedback-based improvement: In Figure 14 we show the effect of the golden-configuration-based feedback to the profiler in order to improve its output. We use a 350 query sample for the QMSUM and KG RAG FinSec dataset as the workload. We see that with the feedback mechanism (blue line), the F1-score improves by 4 – 6% as compared to

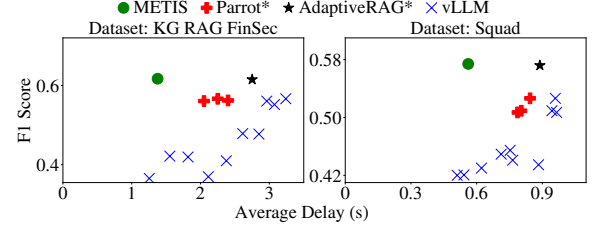


Figure 17. METIS' performance gains remain substantial even with a smaller, open-source LLM profiler.

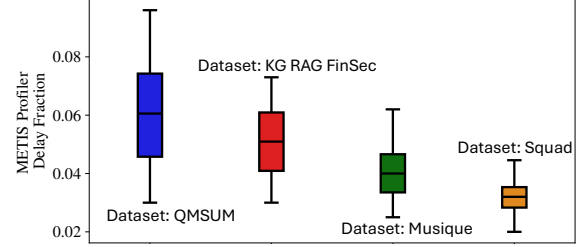


Figure 18. METIS' profiler delay is at most 1/10th of end-to-end response delay across queries from all datasets.

not having feedback (red line) from the outputs of the golden configuration. We ensure that the feedback mechanism cannot result in the output of very expensive configurations, as METIS' joint scheduler will not pick increasingly expensive configurations based on the GPU resource constraint.

7.4 Sensitivity analysis

Changing the inference LLM: Figure 15 shows the outcome of changing the inference LLM to a larger LLM (Llama3.1-70B) on the Musique and QMSUM datasets. Even with a more powerful LLM, METIS achieves 2.1 – 2.4 $\times$ lower delay than *AdaptiveRAG** at a similar F1-score. The best fixed-configuration baselines such as *Parrot** and *vLLM* have a lower F1-score of 7 – 10%. In RAG, models mainly rely on the external context to answer the question instead of the model weights and we only get a 2% improvement in F1-score compared to the smaller inference models.

Incrementally tuning knobs in METIS: In Figure 16, we show the benefit we the improvement we get by incrementally adding more knobs to METIS. We measure this for the QMSUM dataset with the original Mistral-7B-v3 model. We first only tune the `num_chunks` (red point). Progressively we tune the RAG-configuration knobs of `synthesis_method` and `intermediate_length` and `scheduling`. We achieve 5, 4, 3% higher F1-Score compared to *vLLM*. Finally, by adding the `scheduling`, 2.8 $\times$ lower delay reduction in delay.

Changing the profiler LLM: Figure 17 shows the effect of changing the LLM profiler from GPT-4o to a smaller Llama3.1-70B model. METIS with the new profiler, still achieves 1.4 – 2.1 $\times$ over *AdaptiveRAG** with a similar F1-score. Static configurations of *Parrot** and *vLLM* which achieve similar delay, METIS achieves 10 – 14% higher F1-score.

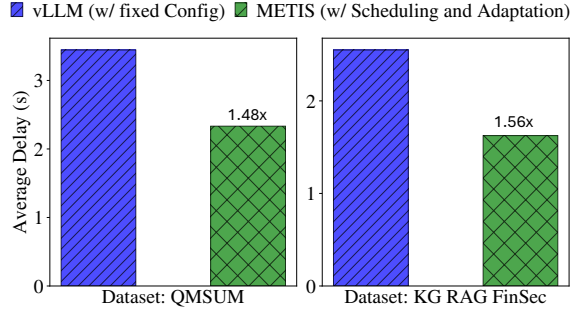


Figure 19. METIS’ under low load without batching

Delay overhead of METIS’s per-query profiling: We show the negligible delay overhead of using an LLM profiler within METIS. Figure 18 shows the fraction of METIS’ profiler of the total end-to-end delay. Using the profiler at most adds 0.1 fraction and in the average case only adds 0.03–0.06 fraction to the total delay across queries from all datasets.

METIS’ performance under low load: In Figure 19, we evaluate METIS under low load by sending queries sequentially, with every query sent after the previous query is completed. We compare with vLLM’s fixed configuration, which achieves the highest quality (blue bar). METIS uses its best-fit algorithm to pick the most expensive configuration from the pruned space of configurations. As METIS only picks from configurations relevant to the query profile, it is still able to reduce delay by 1.48 – 1.56× under low load.

8 Related work

Systems for serving RAG: Several systems have been proposed for RAG [2, 17, 35, 37, 40, 43, 48, 59, 82, 93, 96] which focus on improving retrieval using complex, iterative retrieval algorithms or on serving model selection. METIS can work in conjunction with such systems as METIS focuses on optimizing quality and serving latency, independent of how the retrieval algorithm identifies chunks for retrieval.

KV cache storage and retrieval: Storing and reusing KV cache across different requests have been commonly studied in recent work [2, 14, 22, 30, 36, 44, 50, 53, 54, 68, 81, 92, 98]. METIS can work alongside these systems, where instead of retrieving chunks, it can retrieve the KV Caches for generating the output. In RAG, some additional optimizations are needed to combine KV Caches of different chunks that don’t share a common prefix. This is important as the trivial concatenation of KV Caches loses important cross-attention and reasoning between chunks. These optimizations are enabled by KV Cache blending-based approaches [9, 26, 31, 41, 86, 91]. However RAG workloads have a large number of related contexts across queries and storing all the KV Cache is extremely expensive. We do not measure the KV Cache reuse ratio across queries and leave it for future work.

Prefill-Decode Optimizations: Several systems have proposed optimizations to speed-up prefill and decode for LLMs by leveraging unique properties of each phase [3, 11, 38, 70,

80, 88, 100, 102]. Notable techniques include *chunked-prefill* which allows interleaving prefill and decode requests and *disaggregated prefill* which separates compute nodes for prefill and decode. All of these optimizations enable faster generation speed but don’t focus on generation quality. METIS can be applied with such LLM serving systems optimizations.

9 Discussion and Limitations

While METIS’ profiler and configuration mapping algorithm is currently designed to work with commonly deployed RAG QA pipelines, it can be easily extended to generalize across new RAG configurations, workflows and domains.

Agentic RAG: New research directions in RAG [27, 46, 95] have developed pipelines with LLM agents, tool calling and deep *chain-of-thought* to be used for RAG workloads. For an agentic workflow, a key extension for METIS is to profile the query-complexity and break down a query into multiple sub-queries for planning (e.g., how many sub-queries are needed becomes a new configuration knob). METIS complements such workflows and can continue to perform the joint resource allocation for each sub-query.

Multi-modal RAG: Emerging multi-modal LLMs has led to the need for multi-modal RAG [33, 52, 74] and METIS can be extended to complement this. Using the natural-language profile of the query, METIS can profile the different types of data to be retrieved. e.g., A query might ask for a fact and a supporting image and this becomes a new configuration knob. Based on the properties of the new data retrieved, a configuration selection mapping rule can be added to decide which final RAG configuration should be chosen.

Graph-based RAG: Another emerging area is GraphRAG [17, 34, 101] where retrieval is performed by choosing from hierarchical communities (e.g., coarse grained data aggregation vs fine-grained facts). METIS can complement such approaches by using the complexity profile of the query it generates, in order to choose the appropriate depth of community to use, and add this as a configuration knob.

10 Conclusion

This paper introduces METIS, the first system that focuses on optimizing the tradeoffs between response delay and generation quality in RAG, by jointly scheduling RAG queries and adapting key configurations on a per-query basis. Evaluation on four datasets shows that METIS outperforms the state-of-the-art, reducing generation latency by 1.64 – 2.54× without compromising response quality.

Acknowledgment

We thank all the anonymous reviewers and our shepherd Oana Balmau, for their insightful feedback and suggestions. The project is funded by NSF CNS-2146496, CNS-2131826, CNS-2313190, CNS-1901466, UChicago CERES Center and a Google Faculty Research Award.

References

- [1] Hyperparameter Optimization for RAG. https://docs.llamaindex.ai/en/stable/examples/param_optimizer/param_optimizer/, 2024.
- [2] Reyna Abhyankar, Zijian He, Vikranth Srivatsa, Hao Zhang, and Yiyang Zhang. Infercept: Efficient intercept support for augmented large language model inference. In *Forty-first International Conference on Machine Learning*.
- [3] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 117–134, Santa Clara, CA, July 2024. USENIX Association.
- [4] Cohere AI. Cohere: Cutting-edge gen ai, 2023.
- [5] Jason et al. Ansel. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. In *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*. ACM, April 2024.
- [6] Akari Asai, Sewon Min, Zexuan Zhong, and Danqi Chen. Retrieval-based language models and applications. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 6: Tutorial Abstracts)*, pages 41–46, 2023.
- [7] Angels Balaguer, Vinamra Benara, Renato Luiz de Freitas Cunha, Roberto de M. Estevão Filho, Todd Hendry, Daniel Holstein, Jennifer Marsman, Nick Mecklenburg, Sara Malvar, Leonardo O. Nunes, Rafael Padilha, Morris Sharp, Bruno Silva, Swati Sharma, Vijay Aski, and Ranveer Chandra. Rag vs fine-tuning: Pipelines, tradeoffs, and a case study on agriculture, 2024.
- [8] Harrison Chase. LangChain, October 2022.
- [9] Yihua Cheng, Kuntai Du, Jiayi Yao, and Junchen Jiang. Do large language models need a content delivery network?, 2024.
- [10] Florin Cuconasu, Giovanni Trappolini, Federico Siciliano, Simone Filice, Cesare Campagnano, Yoelle Maarek, Nicola Tonellotto, and Fabrizio Silvestri. The power of noise: Redefining retrieval for rag systems. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR '24*, page 719–729, New York, NY, USA, 2024. Association for Computing Machinery.
- [11] Yinwei Dai, Rui Pan, Anand Iyer, Kai Li, and Ravi Netravali. Apparate: Rethinking early exits to tame latency-throughput tensions in ml serving. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, pages 607–623, 2024.
- [12] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019.
- [13] Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Ruhle, Laks V. S. Lakshmanan, and Ahmed Hassan Awadallah. Hybrid llm: Cost-efficient and quality-aware query routing, 2024.
- [14] Harry Dong, Xinyu Yang, Zhenyu Zhang, Zhangyang Wang, Yuejie Chi, and Beidi Chen. Get more with less: Synthesizing recurrence with kv cache compression for efficient llm inference, 2024.
- [15] José Cassio dos Santos Junior, Rachel Hu, Richard Song, and Yunfei Bai. Domain-driven llm development: Insights into rag and fine-tuning practices. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD '24*, page 6416–6417, New York, NY, USA, 2024. Association for Computing Machinery.
- [16] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The faiss library. 2024.
- [17] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global: A graph rag approach to query-focused summarization, 2024.
- [18] Arvind Neelakantan et al. Text and code embeddings by contrastive pre-training, 2022.
- [19] Hugging Face. Mteb: Massive text embedding benchmark, 2024.
- [20] Wenqi Fan, Yajuan Ding, Liangbo Ning, Shijie Wang, Hengyun Li, Dawei Yin, Tat-Seng Chua, and Qing Li. A survey on rag meeting llms: Towards retrieval-augmented large language models. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD '24*, page 6491–6501, New York, NY, USA, 2024. Association for Computing Machinery.
- [21] Jia Fu, Xiaoting Qin, Fangkai Yang, Lu Wang, Jue Zhang, Qingwei Lin, Yubo Chen, Dongmei Zhang, Saravan Rajmohan, and Qi Zhang. Autorag-hp: Automatic online hyper-parameter tuning for retrieval-augmented generation. *arXiv preprint arXiv:2406.19251*, 2024.
- [22] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. Cost-efficient large language model serving for multi-turn conversations with cached attention. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, pages 111–126, 2024.
- [23] Luyu Gao and Jamie Callan. Long document re-ranking with modular re-ranker. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR '22*, page 2371–2376. ACM, July 2022.
- [24] Aude Genevay. From rag to fabric: Lessons learned from building real-world rags at genaia – part 1. Technical report, AWS, 2024.
- [25] Jiahui Geng, Fengyu Cai, Yuxia Wang, Heinz Koepl, Preslav Nakov, and Iryna Gurevych. A survey of confidence estimation and calibration in large language models. In Kevin Duh, Helena Gomez, and Steven Bethard, editors, *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 6577–6595, Mexico City, Mexico, June 2024. Association for Computational Linguistics.
- [26] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khadelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. *Proceedings of Machine Learning and Systems*, 6:325–338, 2024.
- [27] Minghao Guo, Xi Zhu, Jingyuan Huang, Kai Mei, and Yongfeng Zhang. Reagan: Node-as-agent-reasoning graph agentic network, 2025.
- [28] Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. Lightrag: Simple and fast retrieval-augmented generation, 2024.
- [29] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. Ruler: What's the real context size of your long-context language models?, 2024.
- [30] Cunchen Hu, Heyang Huang, Junhao Hu, Jiang Xu, Xusheng Chen, Tao Xie, Chenxi Wang, Sa Wang, Yungang Bao, Ninghui Sun, and Yizhou Shan. Memserve: Context caching for disaggregated llm serving with elastic memory pool, 2024.
- [31] Junhao Hu, Wenrui Huang, Haoyi Wang, Weidong Wang, Tiancheng Hu, Qin Zhang, Hao Feng, Xusheng Chen, Yizhou Shan, and Tao Xie. Epic: Efficient position-independent context caching for serving large language models, 2024.
- [32] Qitian Jason Hu, Jacob Bieker, Xiuyu Li, Nan Jiang, Benjamin Keigwin, Gaurav Ranganath, Kurt Keutzer, and Shriyash Kaustubh Upadhyay. Routerbench: A benchmark for multi-llm routing system, 2024.
- [33] Wenbo Hu, Jia-Chen Gu, Zi-Yi Dou, Mohsen Fayyaz, Pan Lu, Kai-Wei Chang, and Nanyun Peng. Mrag-bench: Vision-centric evaluation for retrieval-augmented multimodal models, 2025.
- [34] Yiqian Huang, Shiqi Zhang, and Xiaokui Xiao. Ket-rag: A cost-efficient multi-granular indexing framework for graph-rag. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2, KDD '25*, page 1003–1012, New York, NY, USA, 2025. Association for Computing Machinery.
- [35] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong Park. Adaptive-RAG: Learning to adapt retrieval-augmented large

- language models through question complexity. In Kevin Duh, Helena Gomez, and Steven Bethard, editors, *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 7036–7050, Mexico City, Mexico, June 2024. Association for Computational Linguistics.
- [36] Wenqi Jiang, Suvinay Subramanian, Cat Graves, Gustavo Alonso, Amir Yazdanbakhsh, and Vidushi Dadu. Rago: Systematic performance optimization for retrieval-augmented generation serving, 2025.
- [37] Wenqi Jiang, Shuai Zhang, Boran Han, Jie Wang, Bernie Wang, and Tim Kraska. Piperag: Fast retrieval-augmented generation via algorithm-system co-design. *arXiv preprint arXiv:2403.05676*, 2024.
- [38] Xuanlin Jiang, Yang Zhou, Shiyi Cao, Ion Stoica, and Minlan Yu. Neo: Saving gpu memory crisis with cpu offloading for online llm inference, 2024.
- [39] Zhengbao Jiang, Frank F. Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. Active retrieval augmented generation, 2023.
- [40] Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Xin Liu, Xuanzhe Liu, and Xin Jin. Ragcache: Efficient knowledge caching for retrieval-augmented generation, 2024.
- [41] Shuowei Jin, Xueshen Liu, Qingzhao Zhang, and Z. Morley Mao. Compute or load kv cache? why not both?, 2024.
- [42] Dongkyu Kim, Byoungwook Kim, Donggeon Han, and Matouš Eibich. Autorag: Automated framework for optimization of retrieval augmented generation pipeline, 2024.
- [43] Kyoungmin Kim, Kijae Hong, Caglar Gulcehre, and Anastasia Ailamaki. The effect of scheduling and preemption on the efficiency of llm inference serving, 2024.
- [44] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.
- [45] Quinn Leng, Jacob Portes, Sam Havens, Matei Zaharia, and Michael Carbin. Long context rag performance of large language models, 2024.
- [46] Yangning Li, Weizhi Zhang, Yuyao Yang, Wei-Chieh Huang, Yaozu Wu, Junyu Luo, Yuanchen Bei, Henry Peng Zou, Xiao Luo, Yusheng Zhao, Chunkit Chan, Yankai Chen, Zhongfen Deng, Yinghui Li, Hai-Tao Zheng, Dongyuan Li, Renhe Jiang, Ming Zhang, Yangqiu Song, and Philip S. Yu. Towards agentic rag with deep reasoning: A survey of rag-reasoning systems in llms, 2025.
- [47] Zhuowan Li, Cheng Li, Mingyang Zhang, Qiaozhu Mei, and Michael Bendersky. Retrieval augmented generation or long-context llms? a comprehensive study and hybrid approach, 2024.
- [48] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. Parrot: Efficient serving of llm-based applications with semantic variable, 2024.
- [49] Chien-Yu Lin, Keisuke Kamahori, Yiyu Liu, Xiaoxiang Shi, Madhav Kashyap, Yile Gu, Rulin Shao, Zihao Ye, Kan Zhu, Stephanie Wang, Arvind Krishnamurthy, Rohan Kadekodi, Luis Ceze, and Baris Kasikci. Telerag: Efficient retrieval-augmented generation inference with lookahead retrieval, 2025.
- [50] Akide Liu, Jing Liu, Zizheng Pan, Yefei He, Gholamreza Haffari, and Bohan Zhuang. Minicache: Kv cache compression in depth dimension for large language models, 2024.
- [51] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.
- [52] Pei Liu, Xin Liu, Ruoyu Yao, Junming Liu, Siyuan Meng, Ding Wang, and Jun Ma. Hm-rag: Hierarchical multi-agent multimodal retrieval augmented generation, 2025.
- [53] Yuhao Liu, Esha Choukse, Shan Lu, Junchen Jiang, and Madan Musuvathi. Droidspeak: Enhancing cross-llm communication, 2024.
- [54] Yuhao Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. Cachegen: Kv cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*, ACM SIGCOMM '24, page 38–56, New York, NY, USA, 2024. Association for Computing Machinery.
- [55] LlamaHub. Docugami knowledge graph retrieval augmented generation (kg-rag) datasets, 2024.
- [56] Xinbei Ma, Yeyun Gong, Pengcheng He, Hai Zhao, and Nan Duan. Query rewriting for retrieval-augmented large language models, 2023.
- [57] Xueguang Ma, Xinyu Zhang, Ronak Pradeep, and Jimmy Lin. Zero-shot listwise document reranking with a large language model, 2023.
- [58] Kelong Mao, Zheng Liu, Hongjin Qian, Fengran Mo, Chenlong Deng, and Zhicheng Dou. RAG-studio: Towards in-domain adaptation of retrieval augmented generation through self-alignment. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 725–735, Miami, Florida, USA, November 2024. Association for Computational Linguistics.
- [59] Noah Martin, Abdullah Bin Faisal, Hiba Eltigani, Rukhsan Haroon, Swaminathan Lamelas, and Fahad Dogar. Llmprox: Reducing cost to access large language models, 2024.
- [60] Rick Merritt. What is retrieval-augmented generation, aka rag? Technical report, NVIDIA, 2024.
- [61] Alireza Mohammadshahi, Arshad Rafiq Shaikh, and Majid Yazdani. Routoo: Learning to route to large language models effectively, 2024.
- [62] Laurent Mombaerts, Terry Ding, Adi Banerjee, Florian Felice, Jonathan Taws, and Tarik Borogovac. Meta knowledge for retrieval augmented large language models, 2024.
- [63] Zooey Nguyen, Anthony Annunziata, Vinh Luong, Sang Dinh, Quynh Le, Anh Hai Ha, Chanh Le, Hong An Phan, Shruti Raghavan, and Christopher Nguyen. Enhancing q&a with domain-specific fine-tuning and iterative reasoning: A comparative study, 2024.
- [64] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M Waleed Kadous, and Ion Stoica. Routellm: Learning to route llms with preference data, 2024.
- [65] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M Waleed Kadous, and Ion Stoica. Routellm: Learning to route llms with preference data, 2025.
- [66] OpenAI. Openai api, 2023.
- [67] Yang Ouyang, Tong Yu, and Wenchu Wang. Context-aware chatbot extension leveraging HTML data and retrieval-augmented generation (RAG). In *Submitted to Tsinghua University Course: Advanced Machine Learning*, 2024. under review.
- [68] Rui Pan, Zhuang Wang, Zhen Jia, Can Karakus, Luca Zancato, Tri Dao, Yida Wang, and Ravi Netravali. Marconi: Prefix caching for the era of hybrid llms. *arXiv preprint arXiv:2411.19379*, 2024.
- [69] Hongjin Qian, Peitian Zhang, Zheng Liu, Kelong Mao, and Zhicheng Dou. Memorag: Moving towards next-gen rag via memory-inspired knowledge discovery, 2024.
- [70] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: A kvcache-centric disaggregated architecture for llm serving, 2024.
- [71] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. SQuAD: 100,000+ questions for machine comprehension of text. In Jian Su, Kevin Duh, and Xavier Carreras, editors, *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392, Austin, Texas, November 2016. Association for Computational Linguistics.

- [72] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks, 2019.
- [73] Grand View Research. Retrieval augmented generation market size, share and trend analysis report by function (document retrieval, recommendation engines), by application (content generation), by deployment (cloud, on-premises), by end use, by region, and segment forecasts, 2025 - 2030, 2024.
- [74] Monica Riedler and Stefan Langer. Beyond text: Optimizing rag with multimodal inputs for industrial applications, 2024.
- [75] Dongyu Ru, Lin Qiu, Xiangkun Hu, Tianhang Zhang, Peng Shi, Shuaichen Chang, Cheng Jiayang, Cunxiang Wang, Shichao Sun, Huanyu Li, Zizhao Zhang, Binjie Wang, Jiarong Jiang, Tong He, Zhiguo Wang, Pengfei Liu, Yue Zhang, and Zheng Zhang. Ragchecker: A fine-grained framework for diagnosing retrieval-augmented generation, 2024.
- [76] Rana Shahout, Cong Liang, Shiji Xin, Qianru Lao, Yong Cui, Minlan Yu, and Michael Mitzenmacher. Fast inference for augmented large language models, 2024.
- [77] Rulin Shao, Jacqueline He, Akari Asai, Weijia Shi, Tim Dettmers, Sewon Min, Luke Zettlemoyer, and Pang Wei Koh. Scaling retrieval-based language models with a trillion-token datastore. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 91260–91299. Curran Associates, Inc., 2024.
- [78] Sebastian Simon, Alina Mailach, Johannes Dorn, and Norbert Siegmund. A methodology for evaluating rag systems: A case study on configuration dependency validation, 2024.
- [79] Aditi Singh, Abul Ehtesham, Saket Kumar, and Tala Talaei Khoei. Agentic retrieval-augmented generation: A survey on agentic rag, 2025.
- [80] Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. Powerinfer: Fast large language model serving with a consumer-grade gpu. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, pages 590–606, 2024.
- [81] Vikranth Srivatsa, Zijian He, Reyna Abhyankar, Dongming Li, and Yiyang Zhang. Preble: Efficient distributed prompt scheduling for llm serving, 2024.
- [82] Xin Tan, Yimin Jiang, Yitao Yang, and Hong Xu. Teola: Towards end-to-end optimization of llm-based applications, 2024.
- [83] Xiaoliang Tang, Qiang Gao, Jian Li, Nan Du, Qi Li, and Sihong Xie. Mba-rag: a bandit approach for adaptive retrieval-augmented generation through question complexity, 2024.
- [84] Harsh Trivedi, Niranjana Balasubramanian, Tushar Khot, and Ashish Sabharwal. MuSiQue: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554, 2022.
- [85] Xiaohua Wang, Zhenghua Wang, Xuan Gao, Feiran Zhang, Yixin Wu, Zhibo Xu, Tianyuan Shi, Zhengyuan Wang, Shizheng Li, Qi Qian, Ruicheng Yin, Changze Lv, Xiaoqing Zheng, and Xuanjing Huang. Searching for best practices in retrieval-augmented generation. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 17716–17736, Miami, Florida, USA, November 2024. Association for Computational Linguistics.
- [86] Zheng Wang, Boxiao Jin, Zhongzhi Yu, and Minjia Zhang. Model tells you where to merge: Adaptive kv cache merging for llms on long-context tasks, 2024.
- [87] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-Art Natural Language Processing. pages 38–45. Association for Computational Linguistics, October 2020.
- [88] Bingyang Wu, Shengyu Liu, Yinmin Zhong, Peng Sun, Xuanzhe Liu, and Xin Jin. Loongserve: Efficiently serving long-context large language models with elastic sequence parallelism. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, SOSP '24, page 640–654, New York, NY, USA, 2024. Association for Computing Machinery.
- [89] Weijian Xie, Xuefeng Liang, Yuhui Liu, Kaihua Ni, Hong Cheng, and Zetian Hu. Weknow-rag: An adaptive approach for retrieval-augmented generation integrating web search and knowledge graphs, 2024.
- [90] Miao Xiong, Zhiyuan Hu, Xinyang Lu, YIFEI LI, Jie Fu, Junxian He, and Bryan Hooi. Can LLMs express their uncertainty? an empirical evaluation of confidence elicitation in LLMs. In *The Twelfth International Conference on Learning Representations*, 2024.
- [91] Jiayi Yao, Hanchen Li, Yuhang Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. Cacheblend: Fast large language model serving for rag with cached knowledge fusion, 2024.
- [92] Lingfan Yu and Jinyang Li. Stateful large language model serving with pensieve. *arXiv preprint arXiv:2312.05516*, 2023.
- [93] Hailin Zhang, Xiaodong Ji, Yilin Chen, Fangcheng Fu, Xupeng Miao, Xiaonan Nie, Weipeng Chen, and Bin Cui. Pqcache: Product quantization-based kv-cache for long context llm inference, 2024.
- [94] Qizheng Zhang, Ali Imran, Enkelelda Bardhi, Tushar Swamy, Nathan Zhang, Muhammad Shahbaz, and Kunle Olukotun. Caravan: Practical online learning of In-Network ML models with labeling agents. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 325–345, Santa Clara, CA, July 2024. USENIX Association.
- [95] Tianjun Zhang, Shishir G. Patil, Naman Jain, Sheng Shen, Matei Zaharia, Ion Stoica, and Joseph E. Gonzalez. Raft: Adapting language model to domain specific rag, 2024.
- [96] Zhihao Zhang, Alan Zhu, Lijie Yang, Yihua Xu, Lanting Li, Pithchaya Mangpo Phothilimthana, and Zhihao Jia. Accelerating iterative retrieval-augmented language model serving with speculation. In *Forty-first International Conference on Machine Learning*.
- [97] Yiyun Zhao, Prateek Singh, Hanoz Bhathena, Bernardo Ramos, Aviral Joshi, Swaroop Gadiyaram, and Saket Sharma. Optimizing LLM based retrieval augmented generation pipelines in the financial domain. In Yi Yang, Aida Davani, Avi Sil, and Anoop Kumar, editors, *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 6: Industry Track)*, pages 279–294, Mexico City, Mexico, June 2024. Association for Computational Linguistics.
- [98] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- [99] Ming Zhong, Da Yin, Tao Yu, Ahmad Zaidi, Mutethia Mutuma, Rahul Jha, Ahmed Hassan Awadallah, Asli Celikyilmaz, Yang Liu, Xipeng Qiu, and Dragomir Radev. QMSum: A New Benchmark for Query-based Multi-domain Meeting Summarization. In *North American Association for Computational Linguistics (NAACL)*, 2021.
- [100] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 193–210, Santa Clara, CA, July 2024. USENIX Association.
- [101] Yingli Zhou, Yaodong Su, Youran Sun, Shu Wang, Taotao Wang, Runyuan He, Yongwei Zhang, Sicong Liang, Xilin Liu, Yuchi Ma, and Yixiang Fang. In-depth analysis of graph-based rag in a unified

framework, 2025.

- [102] Kan Zhu, Yilong Zhao, Liangyu Zhao, Gefei Zuo, Yile Gu, Dedong Xie, Yufei Gao, Qinyu Xu, Tian Tang, Zihao Ye, et al. Nanoflow: Towards optimal large language model serving throughput. *arXiv preprint arXiv:2408.12757*, 2024.
- [103] Kunlun Zhu, Yifan Luo, Dingling Xu, Ruobing Wang, Shi Yu, Shuo Wang, Yukun Yan, Zhenghao Liu, Xu Han, Zhiyuan Liu, and Maosong Sun. Rageval: Scenario specific rag evaluation dataset generation framework, 2024.

A Appendix

The appendix has not been not peer-reviewed.

A.1 Prompt and input syntax for METIS' LLM profiler

We use a simple prompt to provide the metadata to METIS' LLM profiler. We don't perform any prompt tuning or optimizations for this work as the goal of the prompt was only to get binary decisions from the natural language properties of the query.

```

1 f"""
2 For the given query = {get.query()}: Analyse
3 the language and internal structure of
4 the query and provide the following
5 information :
6
7 1. Does it needs joint reasoning across
8 multiple documents or not.
9
10 2. Provide a complexity profile for the
11 query:
12 Complexity: High/Low \n \
13 Joint Reasoning needed: Yes/No \n "
14
15 3. Does this query need input chunks to
16 be summarized and if yes, provide a
17 range in words for the summarized
18 chunks.
19
20 4. How many pieces of information is
21 needed to answer the query?
22
23 database_metadata = {get.metadata()}
24 chunk_size = {get.chunk_size()}
25
26 Estimate the query profile along with the
27 database_metadata and chunk_size to
28 provide the output.
29
30 """

```

The metadata is a single line summary of the content of the database. For example, for KG RAG FinSec , the metadata is derived from the dataset definition.

The chunk_size is chosen based on guidelines RAG literature for different types of RAG tasks [24, 60]. We don't tune this knob as it is fixed when the database is created. Finally in this work, we don't tune the metadata for the dataset, we use the existing summaries.

```

1 def get_metadata():
2
3     metadata = "The_dataset_consists_of_
4               multiple_chunks_of_information_from_
5               Fortune_500_companies_on_financial_
6               reports_from_every_quarter_of_2023._
7               The_chunk_size_is_1024_tokens."
8
9     return metadata

```

Today, RAG QA datasets already have summaries present along with the queries and contexts. In future work , it will be interesting to study how to effectively construct such a metadata for newer datasets. One possible solution could be an LLM summarizer on a set of values from the dataset which opens up further avenues to perform joint scheduling and configuration tuning.

A.2 Changing the embedding algorithm for the vector database in METIS

METIS picks a state-of-art retrieval algorithm *Cohere-embed-v3.0* [4]. Using two other popular retrieval algorithms *All-mpnet-base-v2* [72] and *text-embedding-3-large-256* [18], the F1-score change remained within 1%. The delay has no measurable difference as the retrieval is > 100× faster than LLM synthesis [6].